

Parcial: Aplicación Frontend en React integrada con API REST

Objetivo del parcial

Desarrollar una aplicación frontend utilizando **React** que consuma el **API REST desarrollado en el parcial anterior**, integrando autenticación de usuarios, manejo de rutas públicas y privadas, consumo de servicios backend, manejo de estado y despliegue de la aplicación en línea.

Este parcial se orienta a la integración de componentes **frontend, backend y base de datos** en una aplicación web funcional, aplicando buenas prácticas básicas de calidad y seguridad.

Descripción general

En este parcial, cada estudiante deberá desarrollar una aplicación web utilizando **React**, la cual debe conectarse al API REST construido en el parcial anterior. La aplicación debe permitir que los usuarios interactúen con los datos del sistema mediante una interfaz gráfica funcional, organizada y conectada correctamente con el backend.

El tema de la aplicación será el mismo trabajado en el parcial anterior. Por ejemplo, si el API desarrollado fue para gestión de productos, reservas, tareas, eventos, clientes, inventario u otro tema, el frontend deberá consumir esos mismos endpoints y presentar las funcionalidades principales desde una interfaz construida en React.

La aplicación debe implementar autenticación de usuarios, rutas públicas y privadas, manejo de estado, consumo de datos desde el API, formularios funcionales y despliegue en una plataforma en línea.

Requerimientos obligatorios del proyecto

1. Creación del proyecto en React

El proyecto debe estar desarrollado utilizando **React**. Puede utilizarse Vite, Create React App u otra herramienta equivalente para la creación del proyecto.

La estructura del proyecto debe estar organizada de forma clara. Se recomienda separar componentes, páginas, servicios, rutas y cualquier otro archivo necesario para mantener el código entendible y fácil de mantener.

Ejemplo de estructura sugerida:

```
src/  
├── components/  
├── pages/  
├── services/  
└── routes/
```

```
├─ context/  
├─ App.jsx  
└─ main.jsx
```

2. Consumo del API REST desarrollado anteriormente

La aplicación debe consumir el API REST desarrollado en el parcial anterior.

Debe evidenciarse el uso de peticiones HTTP hacia el backend, por ejemplo:

```
GET      listar registros  
POST     crear registros  
PUT      actualizar registros  
DELETE   eliminar registros
```

No es obligatorio que todas las rutas del API sean utilizadas, pero sí deben implementarse las operaciones principales necesarias para demostrar la integración entre frontend y backend.

El consumo del API puede realizarse con `fetch`, `axios` u otra alternativa similar.

3. Autenticación de usuarios

La aplicación debe implementar un sistema de autenticación funcional utilizando el API del parcial anterior.

Debe incluir como mínimo:

- Pantalla de inicio de sesión.
- Envío de credenciales al backend.
- Recepción y almacenamiento del token o información de sesión.
- Cierre de sesión.
- Control de acceso según si el usuario está autenticado o no.

El token puede almacenarse en `localStorage`, `sessionStorage` o manejarse mediante estado global, según la estrategia seleccionada por el estudiante.

4. Rutas públicas y privadas

La aplicación debe implementar navegación mediante **React Router** o una alternativa equivalente.

Debe existir una separación clara entre rutas públicas y rutas privadas.

Rutas públicas sugeridas

```
/login  
/register  opcional, si el API lo permite  
/
```

Estas rutas pueden ser vistas sin iniciar sesión.

Rutas privadas sugeridas

```
/dashboard  
/usuarios  
/productos  
/reservas  
/tareas  
/perfil
```

Las rutas privadas solo deben poder visualizarse si el usuario ha iniciado sesión correctamente.

Si un usuario no autenticado intenta acceder a una ruta privada, debe ser redirigido al inicio de sesión.

5. Manejo de estado

La aplicación debe manejar estado de forma adecuada utilizando herramientas propias de React o una alternativa justificada.

Puede utilizarse:

```
useState  
useEffect  
useContext  
useReducer  
Zustand  
Redux Toolkit  
TanStack Query
```

Como mínimo, debe manejarse el estado para:

- Usuario autenticado.
 - Token o sesión activa.
 - Datos obtenidos desde el API.
 - Estados de carga.
 - Mensajes de error o confirmación.
 - Actualización de la interfaz después de crear, editar o eliminar datos.
-

6. Componentes reutilizables

La aplicación debe tener componentes reutilizables que eviten repetir código innecesariamente.

Ejemplos de componentes esperados:

```
Navbar  
Sidebar  
Card  
Table  
Form  
Input  
Button  
ProtectedRoute  
Loading  
Alert
```

No es necesario usar todos estos componentes, pero sí debe evidenciarse una separación lógica entre páginas y componentes reutilizables.

7. Formularios y validación

La aplicación debe incluir formularios funcionales conectados al API.

Los formularios deben permitir, según el tema del proyecto:

- Registrar información.
- Editar información existente.
- Enviar datos al backend.
- Mostrar errores cuando falten campos requeridos.
- Validar entradas básicas antes de enviar la información.

Ejemplos de validación:

```
Campos obligatorios.  
Formato de correo electrónico.  
Longitud mínima de contraseña.  
Valores numéricos válidos.  
Selección obligatoria de una opción.
```

8. Operaciones CRUD desde la interfaz

La aplicación debe permitir administrar al menos una entidad principal del sistema mediante operaciones CRUD.

Debe incluir como mínimo:

- Listar registros.
- Crear registros.
- Editar registros.
- Eliminar registros.
- Mostrar mensajes de éxito o error.

Ejemplo:

Si el API es de productos, la aplicación debe permitir listar productos, crear un producto, editarlo y eliminarlo desde la interfaz.

9. Seguridad básica en el frontend

La aplicación debe aplicar controles básicos de seguridad desde el lado del cliente.

Debe incluir:

- Protección de rutas privadas.
 - No mostrar pantallas privadas a usuarios no autenticados.
 - Enviar el token en las peticiones protegidas cuando el API lo requiera.
 - Cerrar sesión eliminando los datos de autenticación almacenados.
 - Manejar errores de autenticación, por ejemplo token inválido o sesión expirada.
-

10. Despliegue del proyecto

El frontend debe ser desplegado en una plataforma en línea.

Puede utilizarse cualquiera de las siguientes opciones:

```
Vercel  
Netlify  
Render  
Railway  
GitHub Pages  
Otra plataforma equivalente
```

La aplicación desplegada debe conectarse correctamente con el API desplegado del parcial anterior.

No se aceptará únicamente la ejecución local del proyecto como entrega final. El estudiante debe entregar un enlace funcional al frontend desplegado.

Entregables

El estudiante deberá entregar:

1. Enlace al repositorio del proyecto en GitHub.

2. Enlace de la aplicación frontend desplegada.
 3. Enlace del API desplegado utilizado por la aplicación.
 4. Archivo **README.md** con:
 - Nombre del proyecto.
 - Descripción breve.
 - Tecnologías utilizadas.
 - Instrucciones para instalar y ejecutar el proyecto.
 - Variables de entorno necesarias, si aplica.
 - Credenciales de prueba, si aplica.
 - Capturas o explicación breve del funcionamiento.
 5. Evidencia de consumo del API desde la aplicación.
 6. Explicación breve de cómo se manejan las rutas públicas y privadas.
-

Condiciones técnicas

El proyecto debe cumplir con las siguientes condiciones:

- Debe desarrollarse en React.
 - Debe consumir el API REST del parcial anterior.
 - Debe implementar autenticación de usuarios.
 - Debe implementar rutas públicas y privadas.
 - Debe manejar estado en la aplicación.
 - Debe tener formularios funcionales.
 - Debe realizar operaciones CRUD sobre al menos una entidad principal.
 - Debe estar desplegado en línea.
 - Debe tener una estructura de carpetas clara.
 - Debe incluir un **README.md** completo.
 - No se permite simular datos si el API ya proporciona la información.
 - No se permite entregar únicamente capturas de pantalla.
 - No se permite entregar un frontend desconectado del backend.
-

Criterios de evaluación

Criterio	Puntaje
Configuración y estructura del proyecto React	10 pts
Consumo correcto del API REST del parcial anterior	15 pts
Implementación de autenticación de usuarios	15 pts
Implementación de rutas públicas y privadas	15 pts
Manejo adecuado de estado	10 pts
Formularios, validaciones y mensajes al usuario	10 pts
Operaciones CRUD funcionales desde la interfaz	10 pts

Criterio	Puntaje
Despliegue funcional del frontend	10 pts
README, orden del repositorio y claridad de entrega	5 pts
Total	100 pts

Observaciones finales

El parcial no consiste únicamente en diseñar pantallas visuales. El objetivo principal es demostrar la integración funcional entre el frontend y el backend, aplicando una arquitectura básica de aplicación Full Stack.

La aplicación debe permitir que un usuario inicie sesión, navegue por rutas protegidas, consulte información del API, realice operaciones sobre los datos y utilice el sistema desde una versión desplegada en línea.